

Fiche de révision — FP-Tree & FP-Growth

1) Objectif

Trouver les **itemsets fréquents** sans générer tous les candidats (contrairement à Apriori).

Le FP-Tree compresse les transactions en partageant les préfixes communs ; FP-Growth extrait les motifs à partir de cet arbre.

2) Rappel rapide des idées clés

- On calcule d'abord la fréquence (support) de chaque item et on **supprime les items trop rares** ($\text{support} < \text{min_support}$).
 - On **réécrit chaque transaction** en ordonnant ses items par **support global décroissant** (règle importante).
 - On insère ces transactions triées dans un **arbre (FP-Tree)** en augmentant des compteurs.
 - On utilise la **header table** (liste des items fréquents avec pointeurs vers leurs nœuds) pour extraire, pour chaque item, sa **base conditionnelle** puis son **FP-Tree conditionnel** ; on extrait récursivement tous les motifs.
-

3) Exemple complet (pas à pas)

Jeu de transactions (exemple)

Soit la base D de 6 transactions :

```
T1 = {A, B, D}
T2 = {B, C, E}
T3 = {A, B, C, E}
T4 = {B, E}
T5 = {A, B, C, E}
T6 = {A, B, C, D, E}
```

Choix : $\text{min_support} = 3$ (seuil absolu)

3.1 — Comptage initial des items

On parcourt D et compte chaque item :

- A : 4 (T1,T3,T5,T6)
- B : 6 (tous sauf aucun)
- C : 4 (T2,T3,T5,T6)
- D : 2 (T1,T6)
- E : 5 (T2,T3,T4,T5,T6)

→ **Items fréquents** ($\text{support} \geq 3$) : B(6), E(5), A(4), C(4)

→ **Item non fréquent** : D (2) → on l'élimine **dès maintenant**.

Important : un item non fréquent n'apparaîtra **jamais** dans le FP-Tree (principe Apriori).

3.2 — Ordre global des items (décroissant par support)

On ordonne les items fréquents par support décroissant. Ici :

```
ordre global : B (6) > E (5) > A (4) >= C (4)
```

Pour A et C qui ont la même fréquence, on applique une règle de **tie-breaking** (par ex. ordre alphabétique) :
A avant C.

=> ordre final : **B, E, A, C**

3.3 — Nettoyage et tri des transactions

On retire D et on réordonne chaque transaction selon (B,E,A,C):

- $T1 = \{A, B, D\} \rightarrow \text{enlever } D \rightarrow \{A, B\} \rightarrow \text{ordonner} \rightarrow \{B, A\}$
- $T2 = \{B, C, E\} \rightarrow \{B, E, C\}$
- $T3 = \{A, B, C, E\} \rightarrow \{B, E, A, C\}$
- $T4 = \{B, E\} \rightarrow \{B, E\}$
- $T5 = \{A, B, C, E\} \rightarrow \{B, E, A, C\}$
- $T6 = \{A, B, C, D, E\} \rightarrow \text{enlever } D \rightarrow \{A, B, C, E\} \rightarrow \{B, E, A, C\}$

3.4 — Construction du FP-Tree (insertion transaction par transaction)

On initialise : racine (root) vide.

Insertion $T1 = \{B, A\}$

```
root
└─ B:1
   └─ A:1
```

Insertion $T2 = \{B, E, C\}$

- B sous root existe → incrémente B:2
- sous B, A existe (A:1) — mais la transaction veut B→E, il n'y a pas E sous B → créer B→E:1 → puis E→C:1

```
root
└─ B:2
   └─ A:1
      └─ E:1
         └─ C:1
```

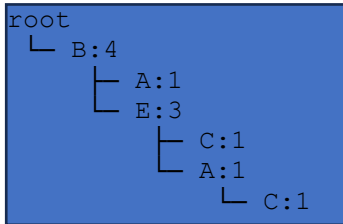
Insertion $T3 = \{B, E, A, C\}$

- root→B exists → B:3
- sous B → E exists → E:2
- sous E → il y a C:1 but transaction next item is A → create E→A:1 → under that create A→C:1

```
root
└─ B:3
   └─ A:1
      └─ E:2
         └─ C:1
            └─ A:1
               └─ C:1
```

Insertion T4 = {B, E}

- root → B:4
- B → E: increment → E:3
(no further child)



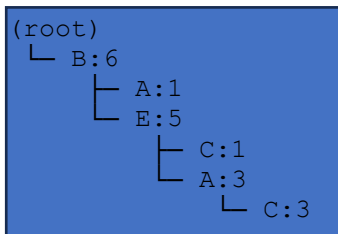
Insertion T5 = {B, E, A, C}

- B:5
- B → E:4
- E → A: increment → A:2
- E → A → C: increment → C:2

Insertion T6 = {B, E, A, C}

- B:6
- B → E:5
- E → A:3
- E → A → C:3

FP-Tree final (ASCII simplifié)



Header table (liens entre nœuds pour chaque item)

- B → node B:6 (sous root)
 - E → node E:5 (sous B)
 - A → nodes B → A:1 and B → E → A:3 (header A pointe à ces nœuds chaînés)
 - C → nodes B → E → C:1 and B → E → A → C:3
-

4) Extraction d'un motif (FP-Growth) — exemple pour l'item C

But : extraire tous les itemsets fréquents contenant C (i.e. suffixe C) en construisant la base conditionnelle de C et son FP-Tree conditionnel.

4.1 — Trouver tous les chemins conduisant aux nœuds C (via header table)

Nœuds C trouvés :

- $\text{node1} = B \rightarrow E \rightarrow C$ (count = 1) $\rightarrow$ préfixe (chemin vers root sans C) = [B, E] avec poids 1
- $\text{node2} = B \rightarrow E \rightarrow A \rightarrow C$ (count = 3) $\rightarrow$ préfixe = [B, E, A] avec poids 3

Base conditionnelle de C (liste de préfixes avec leur compte) :

{ [B, E] : 1, [B, E, A] : 3 }

4.2 — Construire FP-Tree conditionnel pour C

On regroupe et compte les items dans ces préfixes :

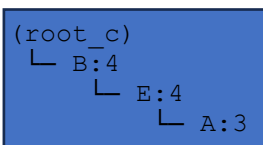
- B : 1 + 3 = 4
- E : 1 + 3 = 4
- A : 3

Avec $\text{min_support} = 3$, on conserve **B (4), E (4), A (3)** (tous ≥ 3).

Maintenant on construit l'arbre conditionnel avec les préfixes ordonnés par fréquence **dans la base conditionnelle** ($B \geq E \geq A$):

- Insert [B, E] count 1 $\rightarrow B:1 \rightarrow E:1$
- Insert [B, E, A] count 3 $\rightarrow B:4 \rightarrow E:4 \rightarrow A:3$

Conditional FP-Tree de C (unique chemin) :



4.3 — Extraction des patterns depuis le conditional tree (cas « single path »)

Quand l'arbre conditionnel est une **single path**, on peut générer **toutes les combinaisons** des items de la voie + le suffixe (ici C). Les supports s'expriment par le compteur minimal le long du chemin correspondant :

Les patterns contenant C :

- {C} — support = support(C) = 4 (on le lit dans header)
- {A, C} — support = 3 (A:3)
- {E, C} — support = 4 (E:4)
- {B, C} — support = 4 (B:4)
- {B, E, C} — support = 4 (E:4 ou B:4)
- {B, E, A, C} — support = 3 (A:3)

(on calcule précisément via les comptes du chemin — le support d'un pattern suffixé par C est la **min** des counts des nœuds nécessaires)

5) Règles d'extraction & suite

- Après avoir extrait tous les itemsets fréquents (pour chaque item en dessinant sa base conditionnelle et son tree conditionnel), on peut générer les **règles d'association** ($A \Rightarrow B$) à partir des itemsets fréquents en calculant la **confiance** = $\text{support}(A \cup B) / \text{support}(A)$.
-

6) Cas particuliers et remarques importantes

a) Items en dessous du seuil (filtrage précoce)

- **On supprime** ces items avant création du FP-Tree.
- Si une transaction ne contient plus d'items (après suppression), on l'ignore.
- Raison : un item non fréquent ne peut être dans aucun itemset fréquent (principe Apriori).

b) Ordre des transactions vs ordre dans transaction

- **Ordre des transactions (T1, T2, ...)** : *indifférent* (l'arbre final sera identique, uniquement l'ordre d'insertion des nœuds peut changer).
- **Ordre des items dans chaque transaction** : **très important** — il faut respecter l'ordre global (support décroissant) pour maximiser le partage de préfixes et la compression.

c) Items ayant mêmes fréquences (ties)

- En cas d'égalité de support, **choisir une règle de tie-breaking** (ex : alphabétique). Il faut appliquer la même règle partout sinon l'arbre perd sa régularité.

d) Arbre « single path »

- Si l'arbre devient une seule longue branche (peu de bifurcations), FP-Growth peut **générer directement toutes les combinaisons** des items sur la branche (accélération).

e) Arbre très « large » (peu de préfixes communs)

- PF-Tree peut ne pas compresser bien si les transactions ont peu de chevauchement → l'arbre peut devenir volumineux en mémoire.

f) Doublons de transactions

- Si la même transaction apparaît plusieurs fois, on l'insère avec un **poids** égal au nombre d'occurrences (utile pour la compression).

g) Mise à jour incrémentale

- FP-Tree n'est pas trivial à mettre à jour incrémentalement ; on peut soit reconstruire l'arbre, soit employer des techniques spécifiques d'update.

7) Pseudo-code (simplifié) — construction et FP-Growth

Construction FP-Tree (pseudo)

```
Input: D (transactions), min_support
Output: FP-Tree root, header_table

1. compter support de chaque item dans D
2. supprimer items whose support < min_support
3. trier items restants par support décroissant (tie-break stable)
4. header_table = liste items triés + pointeurs null
5. root = new Node(null)
6. pour chaque transaction t dans D:
    t' = filtrer t pour ne garder que items fréquents, ordonnés par header_table
    current = root
    pour chaque item x dans t':
        if current has child with label x:
            child.count += 1
        else:
            child = new Node(x, count=1, parent=current)
            ajouter child aux children de current
            ajouter child dans la chaîne header_table[x] (via links)
        current = child
7. retourner (root, header_table)
```

FP-Growth (récursif - simplifié)

```
FP-Growth(header_table, prefix):
    pour chaque item i dans header_table en ordre croissant (moins fréquent -> plus fréquent):
        new_pattern = prefix U {i}
        output new_pattern with support = header_table[i].support
        construire base_conditionnelle(i) // via tous les chemins menant aux nœuds de i
        construire FP-tree_cond à partir de cette base conditionnelle (même principe)
        si FP-tree_cond non vide:
            FP-Growth(FP-tree_cond.header_table, new_pattern)
```

8) Comparaison rapide avec Apriori

- **Apriori** : génère candidats (C_k) puis filtre → coûteux en I/O et combinatoire quand beaucoup de candidats.
- **FP-Growth** : compresse la base en un arbre puis scinde la tâche récursivement → évite de générer explicitement tous les candidats, souvent beaucoup plus rapide sur de grandes bases.
- **Quand utiliser ?** FP-Growth est recommandé pour les bases denses/volumineuses ; Apriori peut être acceptable sur petits jeux ou pour l'apprentissage.

9) Conseils d'implémentation pratiques

- Représenter chaque nœud par : (item, count, parent, children_map, node_link) (node_link pour header table).
- Header table : structure chaîne par item pour retrouver rapidement tous les nœuds d'un même item.
- Stocker counts en int et transactions filtrées en listes ordonnées.

- Attention mémoire : si l'arbre dépasse la RAM, il faut segmenter / out-of-core ou reconstruire partielles.
 - Tests unitaires : vérifier que somme des comptes des nœuds d'un item = support global de l'item.
-

10) Résumé (ce qu'il faut retenir)

- **FP-Tree** compresse les transactions en partageant les préfixes — on ne stocke que les items fréquents.
- **On ordonne les items dans chaque transaction** par fréquence décroissante (ordre global).
- **FP-Growth** construit des tree conditionnels et extrait récursivement des patterns — très efficace sur gros volumes.
- **Cas importants** : suppression précoce des items rares, tie-breaking stable, single-path optimisation, header table avec liens.